

PantryPilot System Testing Report

Prepared by	Zixuan Liang
Course	CISC 594: Software Testing Principles and Techniques
Instructor	Dr. Khalid Lateef
Project	PantryPilot - Smart Meal and Grocery Planner
Submission	System Testing Report
Date	August 14, 2026

Executive Summary

This report documents the system-level testing approach for PantryPilot across its current project versions and its web demonstration layer. The report includes the development and test environment, setup instructions, test-case selection methodology, detailed system test procedures, expected results, observed actual results, and final outcome. The tests exercise externally meaningful workflows including release configuration, quality gates, grocery-list generation, allergen blocking, safe recipe approval, pantry-expiry boundary handling, and browser-to-API integration.

All six formal system scenarios passed, and all five web/API regression tests passed. Together, the pytest suite contains 11 passing automated tests. Local execution used Python 3.11.15, Flask 3.1.1, pytest 8.2.2, and Google Chrome 150.0.7871.129 on macOS 26.5.1. The configured CI environment uses GitHub Actions on ubuntu-latest with Python 3.12 and runs both pytest and the six-scenario system-test runner.

Versions Under Test

Table 1
Application versions and scope

Version	Release name	Scope under test	Release evidence
v1.0.0	Plan & Adapt	Configuration baseline, feature flags, quality gates, and grocery-list generation behavior.	Git tag v1.0.0; baseline commit 842ae5e.
v1.1.0	Safety & Grocery Controls	All v1.0.0 behavior plus allergen filtering, safe recipe approval, grocery aggregation, and pantry-expiry boundary logic.	Git tag v1.1.0; merge commit 834e64e.
Unreleased	Web demonstration layer	Browser workflows and Flask API endpoints that expose the controlled v1.1.0 safety, grocery, pantry, and quality behavior.	CHANGELOG [Unreleased]; tests/test_web_app.py.

Development and Test Environment

Table 2

Software and environment used

Item	Version / value	Use in project
Operating system	macOS 26.5.1 arm64	Local development and report test execution.
Local Python	Python 3.11.15	Executed system_test_runner.py for observed outputs.
CI Python	Python 3.12	Configured in GitHub Actions workflow.
Test framework	pytest 8.2.2	Pinned in requirements.txt and used by CI workflow.
Web framework	Flask 3.1.1	Serves the local demonstration UI and API endpoints.
Browser	Google Chrome 150.0.7871.129	Desktop and responsive presentation-layer verification.
Version control	git version 2.50.1	Controlled baselines, branches, and tags.
Application repository	github.com/liangzixuan/cisc-594	GitHub repository; PantryPilot is maintained in PantryPilot_CM_Generic_Files.
Current application config	baseline_version 1.1.0; release_name Safety & Grocery Controls	Runtime release identity.

Setup Instructions

A maintenance or grading reviewer can recreate the test environment using the following steps:

1. Extract the submitted source-code package and open the PantryPilot_CM_Generic_Files directory.
2. Confirm Git history with `git log --oneline --decorate --graph --all` and verify tags v1.0.0 and v1.1.0.
3. Install the configured test dependency with `python -m pip install -r requirements.txt` if pytest is available.
4. Run automated tests with `python -m pytest`. In CI, GitHub Actions performs this step on Python 3.12.
5. Run the report evidence runner with `python3 system_test_runner.py` to reproduce the actual outputs listed in this report.
6. Start the local web demonstration with `python -m src.web_app`, open `http://127.0.0.1:5000`, and exercise the Plan, Grocery, Pantry, and Quality views.
7. For version-specific review, check out git tag v1.0.0 or v1.1.0 and repeat the relevant test procedures.

Test-Case Selection Methodology

The formal system scenarios were selected from the highest-value user-visible behavior and the highest-risk requirements in the PantryPilot project. The goal was not to test every function in isolation; it was to confirm that each release behaves correctly from a system perspective. The six-scenario set therefore includes configuration loading, release identity, quality-gate configuration, grocery aggregation, allergen safety filtering, safe-output acceptance, and pantry expiry boundary behavior. Five additional Flask test-client cases verify that the demonstration interface reaches those same controls through stable HTTP endpoints.

The selected cases provide adequate coverage for the current project scope because they exercise every major deterministic behavior in the submitted baseline and then verify its presentation through the web API. The allergen and expiry cases cover safety-critical negative and boundary behavior; the grocery case covers user-visible output correctness; configuration and quality-gate cases verify release readiness; web cases cover route availability, request/response integration, and evidence reporting. Manual browser checks at desktop and mobile widths supplement automation for layout and interaction quality. V2 nutrition optimization, external recipe ingestion, and generative-AI behavior remain future scope and are not represented as implemented features.

System Test Plan and Results

Table 3

System test case overview

Test ID	Version	Feature	System behavior tested	Status
V1-ST-01	v1.0.0 / v1.1.0	Release configuration	The system displays the configured application name, version, and release name.	PASS
V1-ST-02	v1.0.0 / v1.1.0	Quality gates	All four quality gates are enabled.	PASS
V1-ST-03	v1.0.0 / v1.1.0	Grocery list generation	Duplicate rice entries are normalized and summed to one grocery item with quantity 3.0 cup.	PASS
V1.1-ST-01	v1.1.0	Allergen safety filter	The system detects soy and blocks the recipe as unsafe.	PASS
V1.1-ST-02	v1.1.0	Safe recipe approval	The system returns True because no declared allergen or alias appears in the recipe.	PASS
V1.1-ST-03	v1.1.0	Pantry expiry boundary	Today and +3 days are included in the near-expiry window; yesterday and +4 days are excluded.	PASS

Table 4*Web presentation-layer regression tests*

Test ID	Workflow	Expected system behavior	Status
WEB-01	Demo page and release data	GET / and GET /api/demo-data return the interface and controlled v1.1.0 release identity.	PASS
WEB-02	Allergen decision endpoint	POST /api/check-recipe blocks the tofu recipe when the household profile includes soy.	PASS
WEB-03	Grocery aggregation endpoint	POST /api/grocery-list consolidates shared ingredients and returns canonical totals.	PASS
WEB-04	Pantry boundary endpoint	POST /api/pantry-status applies the inclusive three-day window and identifies expired items.	PASS
WEB-05	Quality evidence endpoint	GET /api/quality-summary executes and reports all six release scenarios as passing.	PASS

Table 5*V1-ST-01 detailed system test procedure*

Version	v1.0.0 / v1.1.0
Feature	Release configuration
Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.
Test procedure	From the project root, load config/app_config.json and call describe_release(load_config()).
Expected result	The system displays the configured application name, version, and release name.
Actual result	PantryPilot 1.1.0: Safety & Grocery Controls
Status	PASS

Table 6*V1-ST-02 detailed system test procedure*

Version	v1.0.0 / v1.1.0
Feature	Quality gates
Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.

Test procedure	Open the runtime configuration and inspect quality_gates for unit_tests_required, pull_request_required, review_required, and ci_required.
Expected result	All four quality gates are enabled.
Actual result	ci_required, pull_request_required, review_required, unit_tests_required
Status	PASS

Table 7

V1-ST-03 detailed system test procedure

Version	v1.0.0 / v1.1.0
Feature	Grocery list generation
Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.
Test procedure	Create two recipe inputs that both contain rice in cups, then call aggregate_grocery_items(recipes).
Expected result	Duplicate rice entries are normalized and summed to one grocery item with quantity 3.0 cup.
Actual result	[{'name': 'rice', 'quantity': 3.0, 'unit': 'cup'}]
Status	PASS

Table 8

V1.1-ST-01 detailed system test procedure

Version	v1.1.0
Feature	Allergen safety filter
Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.
Test procedure	Create a generated recipe containing 'soy lecithin' and a user profile declaring soy allergy. Run detect_allergen_conflicts and recipe_is_safe_for_profile.
Expected result	The system detects soy and blocks the recipe as unsafe.
Actual result	{'conflicts': ['soy'], 'safe': False}
Status	PASS

Table 9

V1.1-ST-02 detailed system test procedure

Version	v1.1.0
Feature	Safe recipe approval

Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.
Test procedure	Create a recipe with rice, broccoli, and olive oil for a profile with soy and peanut allergies. Run recipe_is_safe_for_profile.
Expected result	The system returns True because no declared allergen or alias appears in the recipe.
Actual result	True
Status	PASS

Table 10
V1.1-ST-03 detailed system test procedure

Version	v1.1.0
Feature	Pantry expiry boundary
Preconditions	Repository extracted; Python can import src.pantrypilot_app; configuration file is available. For v1.1 tests, safety and grocery functions are present on tag/main v1.1.0.
Test procedure	Evaluate is_near_expiry for today, +3 days, +4 days, and yesterday using Aug. 14, 2026 as the current date.
Expected result	Today and +3 days are included in the near-expiry window; yesterday and +4 days are excluded.
Actual result	{'today': True, 'plus_3': True, 'plus_4': False, 'yesterday': False}
Status	PASS

Execution Summary

Table 11
Test execution outcome summary

Version	Tests executed	Passed	Errors / defects observed
v1.0.0 / v1.1.0 shared baseline	3	3	None observed.
v1.1.0 Safety & Grocery Controls	3	3	None observed.
Unreleased web/API layer	5	5	None observed.
Total automated pytest suite	11	11	None observed.

Limitations and Future System Tests

The current tests cover the implemented release baseline and the local demonstration layer; they do not claim that unavailable future features have been tested. The browser review was manual, while HTTP behavior was automated with Flask's test client. Nutrition optimization, live AI-generated recipes, external recipe API ingestion, authentication, persistence, and full pantry-aware planning remain future scope. Those features will require recorded provider responses, security and privacy tests, infeasible-target cases, multi-profile household tests, and full end-to-end deployment checks.

Conclusion

The PantryPilot system testing process demonstrates that the tagged application baseline and its web presentation layer can be installed, identified, configured, and exercised at the system level. Six of six formal scenarios and 11 of 11 automated pytest tests produced the expected outputs with no observed defects. The evidence covers the implemented release behavior, the browser-to-API path, and the most important user-facing and risk-driven controls without overstating unimplemented V2 scope.

References

- Liang, Z. (2026a). PantryPilot configuration management report [Course assignment]. Harrisburg University of Science and Technology.
- Liang, Z. (2026b). PantryPilot risk management report [Course assignment]. Harrisburg University of Science and Technology.
- Liang, Z. (2026c). PantryPilot source repository and system test runner [Course project files]. Harrisburg University of Science and Technology.